

```
def boolean: Gen[Boolean]
def listOfN[A](n: Int, g: Gen[A]): Gen[List[A]]
```

Generates lists of length *n* using the generator *g*

As we discussed in chapter 7, we're interested in understanding what operations are *primitive* and what operations are *derived*, and in finding a small yet expressive set of primitives. A good way to explore what is expressible with a given set of primitives is to pick some concrete examples you'd like to express, and see if you can assemble the functionality you want. As you do so, look for patterns, try factoring out these patterns into combinators, and refine your set of primitives. We encourage you to stop reading here and simply *play* with the primitives and combinators we've written so far. If you want some concrete examples to inspire you, here are some ideas:

- If we can generate a single `Int` in some range, do we need a new primitive to generate an `(Int, Int)` pair in some range?
- Can we produce a `Gen[Option[A]]` from a `Gen[A]`? What about a `Gen[A]` from a `Gen[Option[A]]`?
- Can we generate strings somehow using our existing primitives?

The importance of play

You don't have to wait around for a concrete example to force exploration of the design space. In fact, if you rely exclusively on concrete, obviously useful or important examples to design your API, you'll often miss out on aspects of the design space and generate APIs with ad hoc, overly specific features. We don't want to *overfit* our design to the particular examples we happen to think of right now. We want to reduce the problem to its essence, and sometimes the best way to do this is *play*. Don't try to solve important problems or produce useful functionality. Not right away. Just experiment with different representations, primitives, and operations, let questions naturally arise, and explore whatever piques your curiosity. ("These two functions seem similar. I wonder if there's some more general operation hiding inside," or "Would it make sense to make this data type polymorphic?" or "What would it mean to change this aspect of the representation from a single value to a `List` of values?") There's no right or wrong way to do this, but there are so many different design choices that it's impossible *not* to run headlong into fascinating questions to play with. It doesn't matter where you begin—if you keep playing, the domain will inexorably guide you to make all the design choices that are required.

8.2.4 Generators that depend on generated values

Suppose we'd like a `Gen[(String, String)]` that generates pairs where the second string contains only characters from the first. Or that we had a `Gen[Int]` that chooses an integer between 0 and 11, and we'd like to make a `Gen[List[Double]]` that then generates lists of whatever length is chosen. In both of these cases there's a dependency—we